

Guía de Expresiones Regulares en Linux

1. Introducción

Las expresiones regulares (regex o regexp) son patrones utilizados para buscar y manipular texto. En Linux, son fundamentales para trabajar con archivos y flujos de texto, especialmente en herramientas como `grep`, `sed`, `awk` y editores como `vim`.

¿Por qué son útiles?

- **Búsqueda rápida:** Encuentra patrones específicos en grandes volúmenes de texto.
- **Edición automatizada:** Modifica texto de manera eficiente y repetitiva.
- **Validación de datos:** Verifica formatos como correos electrónicos o números de teléfono.

Herramientas que utilizan regex

- `grep`: Buscar patrones en texto.
- `sed`: Sustituir o manipular texto en flujos.
- `awk`: Procesar y extraer columnas o patrones.
- `vim`: Edición avanzada con regex en modos de búsqueda y reemplazo.

2. Tipos de Expresiones Regulares

En Linux, las expresiones regulares se dividen principalmente en:

Expresiones Regulares Básicas (BRE)

- Usadas por defecto en herramientas como `grep`.

Carácter	Texto buscado
<code>^</code>	Principio de entrada o línea. Ejemplo: <code>grep "^a" ejemplo.txt</code> Muestra las líneas del archivo <code>ejemplo.txt</code> que comienzan por <code>a</code>
<code>\$</code>	Fin de entrada o línea. Ejemplo: <code>grep "s\$" ejemplo.txt</code> Muestra las líneas del archivo <code>ejemplo.txt</code> que terminan por <code>s</code>
<code>*</code>	El carácter anterior 0 o más veces .
<code>[]</code>	Cualquiera de los caracteres entre corchetes. Especifique un rango de caracteres con un guión (por ejemplo, <code>[a-f]</code> es equivalente a <code>[abcdef]</code>).
<code>.</code>	Cualquier carácter individual , salvo el de salto de línea.

Expresiones Regulares Extendidas (ERE)

- Usadas con herramientas como `grep -E` y `sed -r`.
- No necesitan escapado para `?`, `+`, y `{}`.

Carácter	Texto Buscado
<code>?</code>	El carácter anterior una vez como máximo (es decir, indica que el carácter anterior es opcional).
<code>+</code>	El carácter anterior 1 o más veces. Ejemplo: <code>grep -E "d+" ejemplo.txt</code> Muestra las coincidencias <code>d dd ddd dddd</code>
<code>x y</code>	x o y
<code>{n}</code>	Exactamente n apariciones del carácter anterior.
<code>{n,m}</code>	Como mínimo n y como máximo m apariciones del carácter anterior.
<code>[abc]</code>	Cualquiera de los caracteres entre corchetes.
<code>[^abc]</code>	Cualquier carácter que no esté entre corchetes. Especifique un rango de caracteres con un guión (por ejemplo, <code>[^a-f]</code> es equivalente a <code>[^abcdef]</code>).

3. Conceptos Básicos

Literales y caracteres especiales

- Un **literal** coincide exactamente con el texto escrito.
- Los **caracteres especiales** tienen significados específicos: `.`, `*`, `^`, `$`, etc.

Escapado

- Usa `\` para tratar un carácter especial como literal.
 - Ejemplo: `\.` busca un punto literal.

4. Patrones Fundamentales

Coincidencia de caracteres

- `.`: Coincide con cualquier carácter excepto nueva línea.
 - Ejemplo: `h.t` coincide con `hat`, `hot`.

Anclas

- `^`: Inicio de una línea.
 - Ejemplo: `^Hola` coincide con líneas que empiezan con `Hola`.
- `$`: Final de una línea.
 - Ejemplo: `mundo$` coincide con líneas que terminan en `mundo`.

Grupos y rangos

- `[abc]`: Coincide con cualquier carácter dentro de los corchetes.
 - Ejemplo: `[aeiou]` busca vocales.
- `[^abc]`: Coincide con cualquier carácter que **no** esté dentro de los corchetes.
 - Ejemplo: `[^0-9]` busca no dígitos.
- `[a-z]`: Rango de caracteres.
 - Ejemplo: `[a-z]` coincide con cualquier letra minúscula.

Metacaracteres comunes

- `\w`: Coincide con cualquier **carácter de palabra** (letras, dígitos y guion bajo).
- `\d`: Coincide con cualquier **dígito numérico** (del 0 al 9).
- `\s`: Coincide con cualquier **espacio en blanco** (espacios, tabulaciones, saltos de línea).

Repeticiones

- `*`: Coincide con cero o más ocurrencias.
 - Ejemplo: `ho*la` coincide con `hla`, `hola`, `hoola`.
- `+`: Coincide con una o más ocurrencias.
 - Ejemplo: `ho+la` coincide con `hola`, `hoola` (no `hla`).
- `?`: Coincide con cero o una ocurrencia.
 - Ejemplo: `colou?r` coincide con `color` y `colour`.
- `{n,m}`: Coincide entre `n` y `m` ocurrencias.
 - Ejemplo: `a{2,4}` coincide con `aa`, `aaa`, `aaaa`.

5. Grupos y Retroreferencias

Uso de paréntesis para agrupar

- Los paréntesis `( )` definen grupos.
 - Ejemplo: `(ab)+` coincide con `ab`, `abab`, `ababab`.

Referencias hacia atrás

- `\1`, `\2`, etc., se refieren a grupos capturados.
 - Ejemplo: `(\w)\1` encuentra letras repetidas como `aa`, `bb`.
-

6. Expresiones Avanzadas

Alternancia

- `|`: Alterna entre patrones.
 - Ejemplo: `gato|perro` coincide con `gato` o `perro`.

Lookahead y Lookbehind

- **Lookahead**: Asegura que un patrón siga a otro, sin incluirlo en la coincidencia.
 - Ejemplo: `foo(?=bar)` encuentra `foo` solo si es seguido por `bar`.
 - **Lookbehind**: Asegura que un patrón preceda a otro.
 - Ejemplo: `(?<=foo)bar` encuentra `bar` solo si es precedido por `foo`.
-

7. Aplicaciones Prácticas

Buscar y reemplazar con `sed`

- Comando: `sed 's/patrón/reemplazo/' archivo`
 - Ejemplo: `sed 's/gato/perro/' texto.txt` reemplaza `gato` por `perro`.

Filtrar texto con `grep`

- Comando: `grep 'patrón' archivo`
 - Ejemplo: `grep '^Error' log.txt` busca líneas que empiezan con `Error`.

Manipular texto con `awk`

- Comando: `awk '/patrón/ {accion}' archivo`
 - Ejemplo: `awk '/^Usuario/ {print $2}' datos.txt` imprime la segunda columna de líneas que empiezan con `Usuario`.
-

8. Ejemplos Comunes

Validación de correos electrónicos

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

Filtrado de líneas en logs

- Encontrar errores:

```
grep '^[Ee]rror' log.txt
```

Extracción de datos específicos

- Capturar números de teléfono:

```
grep -oE '\b[0-9]{3}-[0-9]{3}-[0-9]{4}\b' archivo.txt
```
